트랜스포머 개요 (Transformer Overview)

학습 목표

- 트랜스포머 등장 배경과 기존 모델 대비 장점 이해
- 전체 아키텍처 구조와 핵심 구성 요소 파악
- 셀프 어텐션, 포지셔널 인코딩, 멀티헤드 어텐션 등 핵심 개념 이해

---

1. 트랜스포머의 등장 배경

1.1 기존 모델들의 한계

순환 신경망(RNN)과 LSTM의 문제점:

1. **순차적 처리의 한계**: RNN과 LSTM은 시퀀스를 순차적으로 처리해야 하므로 병렬화가 어렵습니다.
2. **장거리 의존성 문제**: 긴 시퀀스에서 초기 정보가 손실되는 그래디언트 소실 문제가 여전히 존재합니다.
3. **계산 효율성**: 순차적 처리로 인해 훈련 시간이 오래 걸립니다.

합성곱 신경망(CNN)의 한계:

1. **고정된 수용 영역**: 커널 크기에 의해 제한되는 지역적 패턴만 포착 가능합니다.
2. **장거리 의존성**: 여러 층을 쌓아야만 긴 거리의 관계를 모델링할 수 있습니다.

1.2 트랜스포머의 혁신

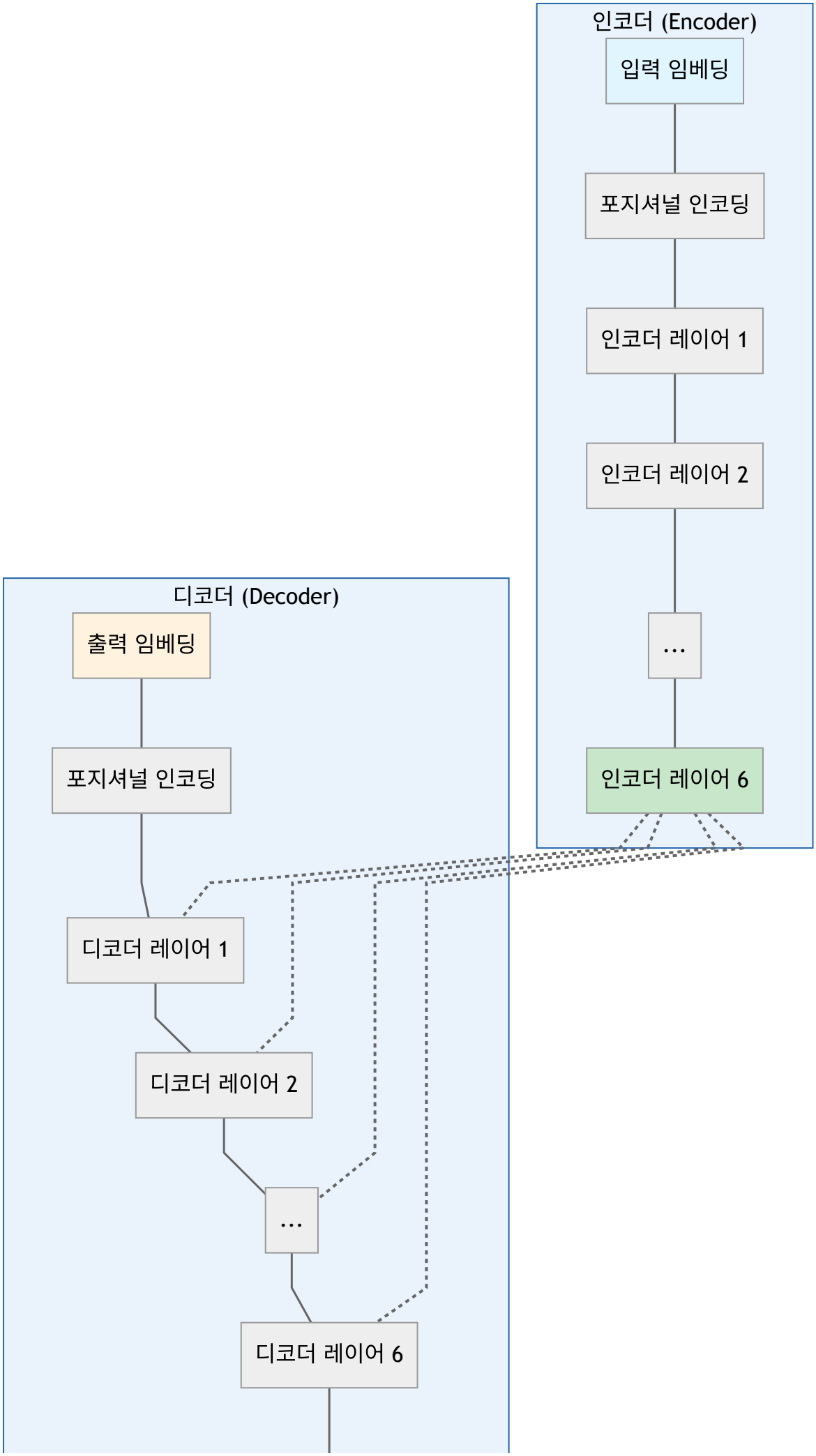
2017년 “Attention Is All You Need” 논문에서 제안된 트랜스포머는 다음과 같은 혁신을 가져왔습니다:

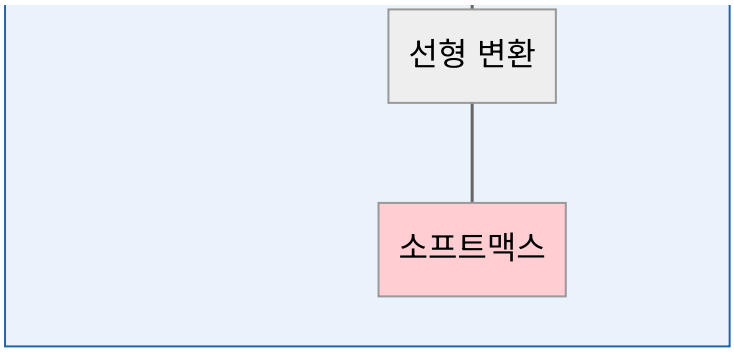
- **완전한 병렬화**: 모든 위치를 동시에 처리할 수 있어 훈련 속도가 크게 향상
- **직접적인 장거리 의존성**: 어텐션 메커니즘을 통해 시퀀스 내 모든 위치 간의 직접적인 연결
- **유연한 표현력**: 다양한 패턴과 관계를 학습할 수 있는 강력한 표현 능력

---

2. 트랜스포머 전체 아키텍처

2.1 아키텍처 다이어그램


인코더 (Encoder)
입력 임베딩
포지셔널 인코딩
인코더 레이어 1
인코더 레이어 2
...
인코더 레이어 6
디코더 (Decoder)
출력 임베딩
포지셔널 인코딩
디코더 레이어 1
디코더 레이어 2
...
디코더 레이어 6



2.2 인코더-디코더 구조

인코더 (Encoder):

- 입력 시퀀스를 고정된 크기의 표현으로 변환
- 6개의 동일한 레이어로 구성
- 각 레이어는 멀티헤드 셀프 어텐션과 피드포워드 네트워크로 구성

디코더 (Decoder):

- 인코더의 출력과 이전에 생성된 출력을 바탕으로 다음 토큰 예측
- 6개의 동일한 레이어로 구성
- 마스크드 셀프 어텐션, 인코더-디코더 어텐션, 피드포워드 네트워크로 구성

---

3. 핵심 구성 요소

3.1 어텐션 메커니즘 (Attention Mechanism)

핵심 아이디어:

어텐션은 입력 시퀀스의 모든 위치에 대해 "얼마나 주목해야 하는가"를 학습하는 메커니즘입니다.

주요 특징:

- **Query (쿼리)**: 현재 처리하고 있는 위치의 정보
- **Key (키)**: 참조할 수 있는 모든 위치의 정보

- **Value (밸류)**: 실제로 가져올 정보

```
# 어텐션의 기본 개념 (의사코드)
def attention(query, key, value):
# 1. 쿼리와 키의 유사도 계산
scores = query @ key.transpose()
# 2. 소프트맥스로 확률 분포 변환
weights = softmax(scores)
# 3. 가중합으로 최종 출력 계산
output = weights @ value
return output
```

3.2 셀프 어텐션 (Self-Attention)

개념:

입력 시퀀스 내에서 각 위치가 같은 시퀀스의 다른 모든 위치와 어떤 관계를 가지는지 학습합니다.

장점:

- 장거리 의존성을 한 번에 포착
- 병렬 처리 가능
- 해석 가능한 어텐션 가중치 제공

3.3 포지셔널 인코딩 (Positional Encoding)

필요성:

트랜스포머는 순차적 처리를 하지 않으므로, 토큰의 위치 정보를 별도로 주입해야 합니다.

구현 방식:

```
# 사인/코사인 함수를 이용한 포지셔널 인코딩

def positional_encoding(position, d_model):

angle_rates = 1 / (10000 ** (2 * (i // 2) / d_model))

# 짝수 인덱스: 사인 함수

pos_encoding[position, 2i] = sin(position * angle_rates[i])

# 홀수 인덱스: 코사인 함수

pos_encoding[position, 2i+1] = cos(position * angle_rates[i])
```

3.4 멀티헤드 어텐션 (Multi-Head Attention)

핵심 아이디어:

여러 개의 어텐션 헤드를 병렬로 실행하여 다양한 종류의 관계를 동시에 학습합니다.

장점:

- 각 헤드가 서로 다른 패턴 학습
- 표현력 향상
- 안정적인 학습

```
# 멀티헤드 어텐션의 기본 구조

class MultiHeadAttention:

def __init__(self, d_model, num_heads):

self.num_heads = num_heads

self.d_k = d_model // num_heads

# 각 헤드별 선형 변환
```

```
self.W_q = Linear(d_model, d_model)

self.W_k = Linear(d_model, d_model)

self.W_v = Linear(d_model, d_model)

self.W_o = Linear(d_model, d_model)
```

---

4. 레이어별 상세 구조

4.1 인코더 레이어

각 인코더 레이어는 다음 구성 요소를 포함합니다:

1. **멀티헤드 셀프 어텐션**

- 입력 시퀀스 내 모든 위치 간의 관계 학습

2. **잔차 연결 (Residual Connection)**

- 그래디언트 소실 문제 해결
- 학습 안정성 향상

3. **레이어 정규화 (Layer Normalization)**

- 각 레이어의 출력을 정규화하여 학습 안정화

4. **피드포워드 네트워크**

- 두 개의 선형 변환과 ReLU 활성화 함수
- 비선형성 추가

4.2 디코더 레이어

디코더 레이어는 인코더보다 복잡한 구조를 가집니다:

1. **마스크드 멀티헤드 셀프 어텐션**

- 미래 토큰을 보지 못하도록 마스킹
- 자기회귀적 생성을 위한 핵심 메커니즘

2. **인코더-디코더 어텐션**

- 디코더가 인코더의 출력에 어텐션
- 입력과 출력 간의 정렬 학습

3. **잔차 연결과 레이어 정규화**

- 각 서브레이어마다 적용

4. **피드포워드 네트워크**

- 인코더와 동일한 구조

---

5. 트랜스포머의 주요 장점

5.1 병렬화 가능성

- 모든 위치를 동시에 처리
- GPU 활용도 극대화
- 훈련 시간 대폭 단축

5.2 장거리 의존성 처리

- 시퀀스 길이에 관계없이 직접적인 연결
- 정보 손실 최소화
- 복잡한 언어 패턴 학습 가능

5.3 해석 가능성

- 어텐션 가중치를 통한 모델 동작 이해
- 어떤 단어에 주목하는지 시각화 가능
- 디버깅과 분석에 유용

5.4 전이 학습 친화적

- 사전 훈련된 모델의 재사용 용이
- 다양한 태스크에 적용 가능
- BERT, GPT 등 강력한 사전 훈련 모델의 기반

---

6. 실제 응용 분야

6.1 자연어 처리

- **기계 번역**: 원래 목적으로 개발
- **텍스트 요약**: 긴 문서의 핵심 내용 추출
- **질의응답**: 문맥을 이해한 정확한 답변 생성
- **감정 분석**: 텍스트의 감정과 의도 파악

6.2 컴퓨터 비전

- **Vision Transformer (ViT)**: 이미지 분류
- **DETR**: 객체 탐지
- **이미지 캡셔닝**: 이미지 설명 생성

6.3 멀티모달 AI

- **CLIP**: 텍스트-이미지 연결

- **DALL-E**: 텍스트로부터 이미지 생성
- **GPT-4V**: 시각적 이해가 가능한 언어 모델

---

7. 학습 로드맵

이 문서를 시작으로 다음 순서로 학습하시기를 권장합니다:

1. **어텐션 메커니즘** - 트랜스포머의 핵심 동작 원리
2. **포지셔널 인코딩** - 위치 정보 주입 방법
3. **멀티헤드 어텐션** - 병렬 어텐션 처리
4. **인코더 아키텍처** - 입력 처리 구조
5. **디코더 아키텍처** - 출력 생성 구조

각 문서는 이론적 설명과 함께 실제 PyTorch 구현 코드를 제공하여 완전한 이해를 도울 것입니다.

이해도 확인 질문

1. 트랜스포머가 RNN/LSTM 대비 가지는 주요 장점 3가지는 무엇인가요?
2. 셀프 어텐션에서 Query, Key, Value의 역할을 각각 설명해보세요.
3. 포지셔널 인코딩이 필요한 이유는 무엇인가요?
4. 멀티헤드 어텐션에서 여러 헤드를 사용하는 이유는 무엇인가요?
5. 인코더와 디코더의 주요 차이점은 무엇인가요?

다음 단계

이제 트랜스포머의 전체적인 구조를 이해했으니, 다음 문서에서는 트랜스포머의 핵심인 **어텐션 메커니즘**을 수학적 공식과 코드 구현을 통해 상세히 알아보겠습니다.

다음 문서: 어텐션 메커니즘

네비게이션

- **다음**: 어텐션 메커니즘
- **목차**: README.md
- **전체 목록**: 네비게이션